THE UNIVERSITY OF LAHORE

1. Data Loading and Understanding

```
import pandas as pd

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

df
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.30	1840.75	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	No	No	No	No	Month-to-month	Yes	Electronic check	70.70	151.65	Yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	...	Yes	Yes	Yes	Yes	One year	Yes	Mailed check	84.80	1990.5	No
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	...	Yes	No	Yes	Yes	One year	Yes	Credit card (automatic)	103.20	7362.9	No
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	...	No	No	No	No	Month-to-month	Yes	Electronic check	29.60	346.45	No
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	...	No	No	No	No	Month-to-month	Yes	Mailed check	74.40	306.6	Yes
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Yes	...	Yes	Yes	Yes	Yes	Two year	Yes	Bank transfer (automatic)	105.65	6844.5	No

7043 rows × 21 columns

```
print("Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)
```

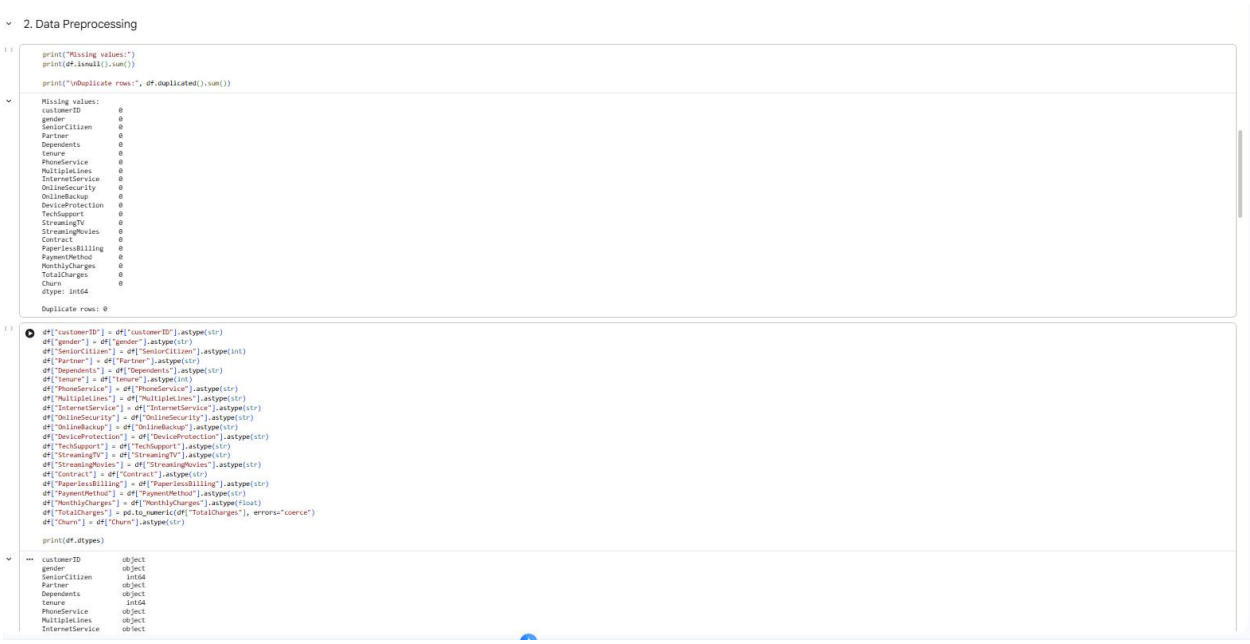
```
Shape: (7043, 21)

Columns:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges', 'TotalCharges

Data Types:
customerID          object
gender              object
SeniorCitizen        int64
Partner             object
Dependents          object
tenure               int64
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
StreamingTV         object
StreamingMovies     object
Contract            object
PaperlessBilling    object
PaymentMethod       object
MonthlyCharges     float64
TotalCharges        object
Churn               object
dtype: object
```

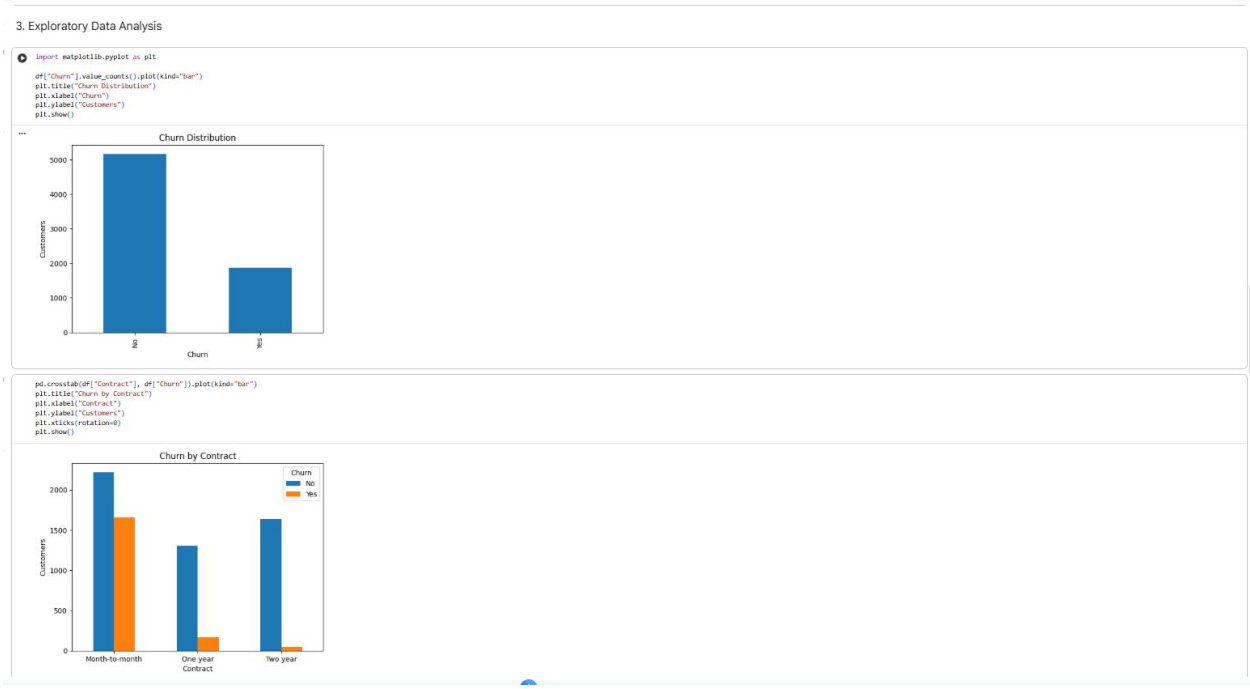
Explanation:

The dataset has 7,043 customers and 21 columns. Churn is the target, Yes means the customer left, No means the customer stayed. Note that Total Charges is stored as text, which needs fixing in preprocessing



Explanation:

The data is now clean, with 7,032 rows and 20 columns, and all columns have corre data types



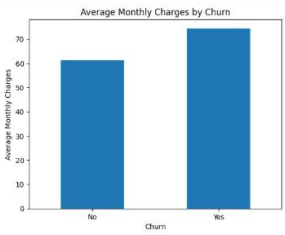
Churn Distribution:

Observation: 5,163 customers stayed and 1,869 left, so about 73% stayed and 27% churned. The data is imbalanced, which affects how we judge the models

Churn by Contract:

Observation: Month-to-month customers churn the most, compared with about 11% for one-year and about 3% for two-year contracts. Longer contracts keep customers.

```
df.groupby("Churn")["MonthlyCharges"].mean().plot(kind="bar")
plt.title("Average Monthly Charges by Churn")
plt.xlabel("Churn")
plt.ylabel("Average Monthly Charges")
plt.xticks(rotation=0)
plt.show()
```



4. Feature Engineering and Encoding

```
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})

X = df.drop("Churn", axis=1)
y = df["Churn"]

X = pd.get_dummies(X, drop_first=True)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (7032, 30)
y shape: (7032,)
```

Average Monthly Charges by Churn:

Observation: Customers who churned pay more on average than customers who stayed. Churners also have a much shorter average tenure.

4. Feature Engineering and Encoding

```
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})

X = df.drop("Churn", axis=1)
y = df["Churn"]

X = pd.get_dummies(X, drop_first=True)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (7032, 30)
y shape: (7032,)
```

X now contains 30 numeric input features for 7,032 customers, and y contains the 0/1 churn label for each customer. The data is ready for training

5. Training and Testing

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (5625, 30)
Testing data: (1407, 30)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

model1 = LogisticRegression(solver="liblinear")
model2 = DecisionTreeClassifier(random_state=42)

model1.fit(X_train, y_train)
model2.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

Both models learned patterns from the training set and are ready to be evaluated on the unseen test set

6. Model Evaluation

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

pred1 = model1.predict(X_test)
pred2 = model2.predict(X_test)

print("Logistic Regression")
print("Accuracy:", accuracy_score(y_test, pred1))
print("Precision:", precision_score(y_test, pred1))
print("Recall:", recall_score(y_test, pred1))
print("F1 Score:", f1_score(y_test, pred1))
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred1))

print("\nDecision Tree")
print("Accuracy:", accuracy_score(y_test, pred2))
print("Precision:", precision_score(y_test, pred2))
print("Recall:", recall_score(y_test, pred2))
print("F1 Score:", f1_score(y_test, pred2))
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred2))

if f1_score(y_test, pred1) > f1_score(y_test, pred2):
    print("\nFinal Model: Logistic Regression")
else:
    print("\nFinal Model: Decision Tree")
```

```
Logistic Regression
Accuracy: 0.7874911158493249
Precision: 0.6229508196721312
Recall: 0.5080213903743316
F1 Score: 0.5596465390279823
Confusion Matrix:
[[918 115]
 [184 190]]

Decision Tree
Accuracy: 0.7249466950959488
Precision: 0.4838709677419355
Recall: 0.5213903743315508
F1 Score: 0.5019305019305019
Confusion Matrix:
[[825 208]
 [179 195]]

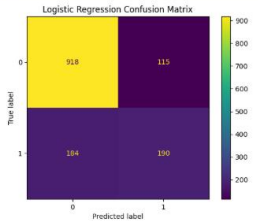
Final Model: Logistic Regression
```

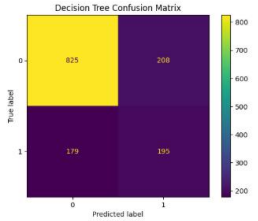
Which model is better and why: Logistic Regression is better. It has higher Accuracy, higher Precision and a higher F1-Score. The Decision Tree has a slightly higher Recall, but it raises many more false alarms. A fully grown Decision Tree also tends to overfit the training data, which is why it performs worse on new data. Logistic Regression was therefore selected as the final model

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, pred1)
plt.title("Logistic Regression Confusion Matrix")
plt.show()

ConfusionMatrixDisplay.from_predictions(y_test, pred2)
plt.title("Decision Tree Confusion Matrix")
plt.show()
```





7. Prediction

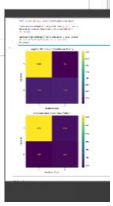
```
sample = X_test.iloc[[0]]

if f1_score(y_test, pred1) > f1_score(y_test, pred2):
    final_model = model1
else:
    final_model = model2

prediction = final_model.predict(sample)[0]

if prediction == 1:
    print("Prediction: Churn")
else:
    print("Prediction: No Churn")
```

```
Prediction: No Churn
```



Explanation:

What the result means: Customer 1 is a new customer (2 months) on a month-to-month contract with fiber optic and electronic check, which is the profile most likely to leave, so the model predicts Churn. Customer 2 is a loyal customer (60 months) on a two-year contract with support services, so the model predicts No Churn. These predictions agree with the patterns seen in the EDA.